NEAR One Foreign Transaction Validation

Security Assessment with Fix Review

June 1, 2026

Prepared for:
Mårten Blankfors
NEAR One

Prepared by: **Fredrik Dahlgren and Joe Doyle**

Table of Contents

Table of Contents **1**
Project Summary **2**
Executive Summary **3**
Project Goals **5**
Project Targets **6**
Project Coverage **7**
Summary of Findings **8**
Detailed Findings **9**
1. Duplicate requests overwrite pending yield promises 9
2. MpcContract::respond_verify_foreign_tx does not check that payload_hash matches the response 12
3. verify_foreign_transaction does not charge properly for storage 15
4. Several voting methods accept cross-contract calls under the participant's signer identity 18
5. Node provider ordering depends only on request ID and local configuration quirks 21
6. API tokens always have a prefix in authentication headers (defaults to "Bearer") 23
A. Vulnerability Categories **25**
B. Fix Review Results **27**
C. Fix Review Status Categories **29**
About Trail of Bits **30**
Notices and Remarks **31**

Project Summary

Contact Information

The following project manager was associated with this project:

Kimberly Espinoza, Project Manager
kimberly.espinoza@trailofbits.com

The following engineering director was associated with this project:

Jim Miller, Engineering Director, Application Security and Cryptography
james.miller@trailofbits.com

The following consultants were associated with this project:

Fredrik Dahlgren, Consultant
fredrik.dahlgren@trailofbits.com

Joe Doyle, Consultant
joe.doyle@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
April 1, 2026	Pre-project kickoff call
April 20, 2026	Delivery of report draft
April 20, 2026	Report readout meeting
April 29, 2026	Delivery of draft comprehensive report
June 1, 2026	Delivery of final comprehensive report

Executive Summary

Engagement Overview

NEAR One engaged Trail of Bits to review the security of their foreign transaction validation protocol.

This protocol allows data from other blockchain systems to be imported into NEAR One by having a quorum of signers query an external data provider, and attest to that provider's response.

A team of two consultants conducted the review from April 9 to April 24, 2026, for a total of 3.5 engineer-weeks of effort. Our testing efforts focused on all paths for how data could enter the system. With full access to source code and documentation, we performed static and dynamic testing of the codebase, using automated and manual processes.

Observations and Impact

All identified issues relate to mishandling API input data, including requests overwriting state (TOB-NEARFTV-1), uneven or missing authentication (TOB-NEARFTV-2, TOB-NEARFTV-3, TOB-NEARFTV-4), and apparently broken defaults (TOB-NEARFTV-5, TOB-NEARFTV-6).

These findings point to a systemic lack of adversarial and edge-case testing of this component, which the NEAR team should address.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that NEAR One take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Document and specify the API and the precise intended behavior for each call.** The `foreign-chain-transactions.md` file is not sufficiently explicit about the checks and requirements of the API itself, and most of these findings were in underspecified portions.
- **Implement property-based testing of all entry points.** Property-based testing to fuzz and find edge cases could have easily found the high-severity TOB-NEARFTV-4, TOB-NEARFTV-5, and TOB-NEARFTV-6.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

Severity	*Count*
High	2
Medium	2
Low	1
Informational	1
Undetermined	0

CATEGORY BREAKDOWN

Category	*Count*
Access Controls	1
Configuration	1
Data Validation	2
Denial of Service	2

Project Goals

The engagement was scoped to provide a security assessment of the NEAR One foreign transaction validation protocol. Specifically, we sought to answer the following non-exhaustive list of questions:

- Are RPC endpoint API-keys redacted on-chain?
- Can protocol participants vote multiple times for the same foreign chain policy?
- Can RPC provider selection be biased or cause a single malicious provider to be selected by all nodes?
- Are foreign chain finality checks implemented correctly?
- Are Ethereum transaction receipts checked correctly to ensure that transactions succeeded?
- Is the transaction execution status checked correctly for Starknet transactions?
- Does the implementation properly protect against presignature reuse?

Project Targets

The engagement involved reviewing and testing the following targets.

near/mpc

Repository	https://github.com/near/mpc
Version	`069ebdaee866791da7a1e00f29f30785cbb7e676`
Type	Rust
Platform	Multiple

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- **Design review.** We reviewed the foreign transaction validation design document to understand the implementation and to identify any weaknesses or opportunities to strengthen the design.
- **Code review.** We manually reviewed the implementation against the specification, looking for deviations and potentially vulnerable behavior.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Duplicate requests overwrite pending yield promises	Denial of Service	**Medium**
2	MpcContract::respond_verify_foreign_tx does not check that payload_hash matches the response	Data Validation	**High**
3	verify_foreign_transaction does not charge properly for storage	Denial of Service	**Low**
4	Several voting methods accept cross-contract calls under the participant's signer identity	Access Controls	**High**
5	Node provider ordering depends only on request ID and local configuration quirks	Configuration	**Medium**
6	API tokens always have a prefix in authentication headers (defaults to "Bearer")	Data Validation	**Informational**

Detailed Findings

1. Duplicate requests overwrite pending yield promises

Severity: **Medium**	Difficulty: **Low**
Type: Denial of Service	Finding ID: TOB-NEARFTV-1
Target: `crates/contract/src/lib.rs`, `crates/near-mpc-contract-interface/src/types/foreign_chain.rs`	

Description

The `verify_foreign_transaction` entrypoint stores pending requests in a `LookupMap` keyed by the `VerifyForeignTransactionRequest` struct, which contains the RPC request parameters, a domain ID, and a payload version. However, the key does not include any caller-specific information. If a second caller submits a request with identical parameters, the `insert` call overwrites the first caller's `YieldIndex`, permanently orphaning its yield promise.

```
fn add_verify_foreign_tx_request(
    &mut self,
    request: &VerifyForeignTransactionRequest,
    data_id: CryptoHash,
) -> bool {
    self.pending_verify_foreign_tx_requests
        .insert(request.clone(), YieldIndex { data_id })
        .is_some()
}
```

Figure 1.1: A second identical request will overwrite the original's pending request data. (mpc/crates/contract/src/lib.rs#L216–L224)

As shown in figures 1.2 and 1.3, the `VerifyForeignTransactionRequest` type used as the map key is entirely caller-independent.

```
pub struct VerifyForeignTransactionRequest {
    pub request: ForeignChainRpcRequest,
    pub domain_id: DomainId,
    pub payload_version: ForeignTxPayloadVersion,
}
```

Figure 1.2: The `VerifyForeignTransactionRequest` type does not hold any caller-specific data.

(*mpc/crates/near-mpc-contract-interface/src/types/foreign_chain.rs#L95–L99*)

```
pub struct EvmRpcRequest {
    pub tx_id: EvmTxId,
    pub extractors: Vec<EvmExtractor>,
    pub finality: EvmFinality,
}
```

Figure 1.3: None of the `ForeignChainRpcRequest` variants hold caller-specific data.
(*mpc/crates/near-mpc-contract-interface/src/types/foreign_chain.rs#L178–L182*)

When MPC nodes later respond via `respond_verify_foreign_tx`, only the most recently stored `data_id` is resolved (figure 1.4). The overwritten caller's yield promise can never be resumed and will time out.

```
// First get the yield promise of the (potentially timed out) request.
if let Some(YieldIndex { data_id }) =
    self.pending_verify_foreign_tx_requests.remove(&request)
{
    // Finally, resolve the promise. This will have no effect if the request already
    // timed.
    env::promise_yield_resume(&data_id, serde_json::to_vec(&response).unwrap());
    Ok(())
} else {
    Err(InvalidParameters::RequestNotFound.into())
}
```

Figure 1.4: Only the last pending request can be resolved in `respond_verify_foreign_tx`.
(*mpc/crates/contract/src/lib.rs#L814–L823*)

The project's design document specifies that `VerifyForeignTransactionRequest` should also include a tweak derived from the caller's `predecessor_id` and `derivation_path`, but this is currently not implemented.

Exploit Scenario

An attacker monitors the NEAR network for `verify_foreign_transaction` calls from bridge relayers. When the attacker observes a pending request, they submit an identical call with the same foreign chain transaction ID, extractors, domain ID, and payload version. Because the map key has no caller-specific component, the attacker's `insert` overwrites the relayer's `YieldIndex`. The relayer's yield promise is orphaned and eventually times out, causing the relayer to lose its deposit and gas. The attacker repeats this at minimal cost (one minimum deposit per attempt) to persistently deny service to bridge operations, stalling cross-chain fund transfers.

Recommendations

Short term, include caller identity in the `LookupMap` key used for `pending_verify_foreign_tx_requests`. This can be done by adding a tweak derived from `(predecessor_id, derivation_path)` to the `VerifyForeignTransactionRequest` struct, as described in the design document.

Long term, ensure that all pending request maps must include caller-binding information in their keys. Add tests that submit identical requests from two different predecessors and assert that both yield promises are independently resolvable. This will prevent similar issues from being introduced if new request types are added.

2. MpcContract::respond_verify_foreign_tx does not check that payload_hash matches the response

Severity: **High**	Difficulty: **Medium**
Type: Data Validation	Finding ID: TOB-NEARFTV-2
Target: `crates/contract/src/lib.rs`	

Description

Foreign transactions are brought into the NEAR system via a request for that data, which is then resolved by calling `respond_verify_foreign_tx`, shown in figure 2.1, as a signed package. However, this call checks only that the response has *a* signature and does not appear to perform any validation that the `payload_hash` field is in fact the hash of any particular data.

```
#[handle_result]
pub fn respond_verify_foreign_tx(
    &mut self,
    request: VerifyForeignTransactionRequest,
    response: VerifyForeignTransactionResponse,
) -> Result<(), Error> {
    let signer = Self::assert_caller_is_signer();
    // ...

    self.assert_caller_is_attested_participant_and_protocol_active();
    // ...

    let domain = request.domain_id;
    let public_key = self.public_key_extended(domain.0.into())?;

    let signature_is_valid = match (&response.signature, public_key) {
        (
            dtos::SignatureResponse::Secp256k1(signature_response),
            PublicKeyExtended::Secp256k1 { near_public_key },
        ) => {
            let secp_pk = dtos::Secp256k1PublicKey::try_from(&near_public_key)
                .expect("Secp256k1 variant always has a secp256k1 key");
            let payload_hash: [u8; 32] = response.payload_hash.0;

            // Check the signature is correct against the root public key
            near_mpc_signature_verifier::verify_ecdsa_signature(
                signature_response,
                &payload_hash,
                &secp_pk,
            )
            .is_ok()
        }
    // ...
```

```
    };
    if !signature_is_valid {
        return Err(RespondError::InvalidSignature.into());
    }

    // First get the yield promise of the (potentially timed out) request.
    if let Some(YieldIndex { data_id }) =
        self.pending_verify_foreign_tx_requests.remove(&request)
    {
        // Finally, resolve the promise. This will have no effect if the request already timed.
        env::promise_yield_resume(&data_id, serde_json::to_vec(&response).unwrap());
        Ok(())
    } else {
        Err(InvalidParameters::RequestNotFound.into())
    }
}
```

Figure 2.1: The pending request is removed and completed based on a hash, not on the response itself. (mpc/crates/contract/src/lib.rs#757–824)

This allows any caller who is allowed to post a signed response to entirely fabricate the actual contents of the response.

The `ForeignChainSignatureVerifier::verify_signature` method shown in figure 2.2 correctly checks the payload hash, so it is possible for downstream consumers to detect and reject fabricated responses, which would make this strictly a denial-of-service attack.

```
    pub fn verify_signature(
        self,
        response: &VerifyForeignTransactionResponse,
        // TODO(#2232): don't use interface API types for public keys
        public_key: &PublicKey,
    ) -> Result<(), VerifyForeignChainError> {
        let expected_payload = ForeignTxSignPayload::V1(ForeignTxSignPayloadV1 {
            request: self.request,
            values: self.expected_extracted_values,
        });
        let expected_payload_hash = expected_payload
            .compute_msg_hash()
            .map_err(|_| VerifyForeignChainError::FailedToComputeMsgHash)?;

        let payload_is_correct = expected_payload_hash == response.payload_hash;

        if !payload_is_correct {
            return Err(VerifyForeignChainError::IncorrectPayloadSigned {
                got: response.payload_hash.clone(),
                expected: expected_payload_hash,
            });
        }
        let verification_result = match (public_key, &response.signature) {
            (
                PublicKey::Secp256k1(secp256k1_public_key),
                SignatureResponse::Secp256k1(k256_signature),
            ) => near_mpc_signature_verifier::verify_ecdsa_signature(
                k256_signature,
                &expected_payload_hash,
```

```
                secp256k1_public_key,
            ),
            (PublicKey::Ed25519(ed25519_public_key), SignatureResponse::Ed25519 { signature })
=> {
                near_mpc_signature_verifier::verify_eddsa_signature(
                    signature,
                    expected_payload_hash.as_slice(),
                    ed25519_public_key,
                )
            }
            // TODO(#2234): improve types so these errors can't happen
            (PublicKey::Bls12381(_bls12381_g2_public_key), _) => {
                return Err(VerifyForeignChainError::UnexpectedSignatureScheme);
            }
            _ => return Err(VerifyForeignChainError::UnexpectedSignatureScheme),
        };

        verification_result.map_err(|_| VerifyForeignChainError::SignatureVerificationFailed)
    }
}
```

Figure 2.2: Correct payload hash checking in `ForeignChainSignatureVerifier` (`mpc/crates/near-mpc-sdk/src/foreign_chain.rs#36–84`)

Exploit Scenario

Alice uses the NEAR foreign transaction service to implement an Ethereum bridge. Bob replaces the contents of a true transaction response with a fabricated transaction, and drains Alice's bridge of funds.

Recommendations

Short term, ensure that the code checks the payload hash against the payload itself.

Long term, always have the code validate the entire data structure a signature authenticates.

3. verify_foreign_transaction does not charge properly for storage

Severity: **Low**	Difficulty: **Low**
Type: Denial of Service	Finding ID: TOB-NEARFTV-3
Target: `crates/contract/src/lib.rs`	

Description

Data stored in the contract costs NEAR, and other method calls on this contract charge the caller proportional to the storage burden, as shown in figure 3.1. However, when `verify_foreign_transaction` (figure 3.2) enqueues a foreign transaction request, no such charge is made.

```
#[payable]
#[handle_result]
pub fn submit_participant_info(
    &mut self,
    proposed_participant_attestation: dtos::Attestation,
    tls_public_key: dtos::Ed25519PublicKey,
) -> Result<(), Error> {
    let proposed_participant_attestation =
        proposed_participant_attestation.try_into_contract_type()?;

    let account_key = env::signer_account_pk();
    let account_id = Self::assert_caller_is_signer();
    // ...

    // Save the initial storage usage to know how much to charge the proposer for the storage
    // used
    let initial_storage = env::storage_usage();

    let tee_upgrade_deadline_duration =
        Duration::from_secs(self.config.tee_upgrade_deadline_duration_seconds);

    // ...

    let attestation_storage_must_be_paid_by_caller =
        is_new_attestation || caller_is_not_participant;

    if attestation_storage_must_be_paid_by_caller {
        let storage_used = env::storage_usage() - initial_storage;
        let cost = env::storage_byte_cost().saturating_mul(storage_used as u128);
        let attached = env::attached_deposit();

        if attached < cost {
            return Err(InvalidParameters::InsufficientDeposit.message(format!(
                "Attached {}, Required {}",
                attached.as_yoctonear(),
                cost.as_yoctonear(),
```

```
            )));
        }

        // Refund the difference if the proposer attached more than required
        if let Some(diff) = attached.checked_sub(cost) {
            if diff > NearToken::from_yoctonear(0) {
                Promise::new(account_id).transfer(diff).detach();
            }
        }
    }

    Ok(())
}
```

Figure 3.1: Storage payment logic in `submit_participant_info` (mpc/crates/contract/src/lib.rs#828–903)

```
   #[handle_result]
    #[payable]
    pub fn verify_foreign_transaction(&mut self, request: VerifyForeignTransactionRequestArgs)
{
 …
        let gas_required =
            Gas::from_tgas(self.config.sign_call_gas_attachment_requirement_tera_gas);

        // Make sure call will not run out of gas doing yield/resume logic
        if env::prepaid_gas() < gas_required {
            env::panic_str(
…
            );
        }

        let predecessor = env::predecessor_account_id();
        require_deposit(MINIMUM_SIGN_REQUEST_DEPOSIT, &predecessor);

        if !self.accept_requests {
            env::panic_str(&TeeError::TeeValidationFailed.to_string())
        }
        let callback_gas = Gas::from_tgas(
            self.config
                .return_signature_and_clean_state_on_success_call_tera_gas,
        );

        let request = args_into_verify_foreign_tx_request(request);

        let promise_index = env::promise_yield_create(
            method_names::RETURN_VERIFY_FOREIGN_TX_AND_CLEAN_STATE_ON_SUCCESS,
            serde_json::to_vec(&(&request,)).unwrap(),
            callback_gas,
            GasWeight(0),
            DATA_ID_REGISTER,
        );

        // Store the request in the contract's local state
        let return_sig_id: CryptoHash = env::read_register(DATA_ID_REGISTER)
            .expect("read_register failed")
            .try_into()
            .expect("conversion to CryptoHash failed");
        if self.add_verify_foreign_tx_request(&request, return_sig_id) {
```

```
            log!("signature request already present, overriding callback.")
        }

        env::promise_return(promise_index);
    }
}
```

Figure 3.2: Requests are added to the request queue without charging storage costs. (mpc/crates/contract/src/lib.rs#518–615)

In addition, several data fields allow for requests with unbounded data to be included, including the following:

- The `extractors` field of the various cases of `ForeignChainRpcRequest`
- The `values` field of `ForeignTxSignPayloadV1`
- The `data` and `topics` fields of `EvmLog`
- The `data` and `keys` field of `StarknetLog`

We have not determined if other mechanisms incidentally limit those fields, but the NEAR team should ensure that they are not available to an attacker who wishes to trigger resource exhaustion.

Exploit Scenario

Alice submits a large number of requests, causing the transaction queue to grow, draining the funds of the contract and causing a denial of service.

Recommendations

Short term, add storage accounting to `verify_foreign_transaction`.

Long term, ensure that cost-imposing operations have a misuse-resistant API that makes it difficult to call them without properly charging.

4. Several voting methods accept cross-contract calls under the participant's signer identity

Severity: **High**	Difficulty: **High**
Type: Access Controls	Finding ID: TOB-NEARFTV-4
Target: `crates/contract/src/lib.rs`, `crates/contract/src/primitives/key_state.rs`, `crates/contract/src/state/{running,resharing,initializing}.rs`	

Description

NEAR distinguishes the signer of a transaction (the account whose key signed it) from the predecessor of an internal call (the immediate caller, which may be another contract that the signer's outer transaction invoked). A method that authenticates by signer alone is callable through any contract the signer happens to invoke; the standard mitigation is to additionally assert `env::signer_account_id() == env::predecessor_account_id()`.

The contract has a helper for exactly this: `Self::assert_caller_is_signer()`, folded into `voter_or_panic()`. However, the lower-level type constructors `AuthenticatedAccountId::new` and `AuthenticatedParticipantId::new`, such as shown below in figure 4.1, only check participant-set membership against `env::signer_account_id()`. They do not check that `signer == predecessor`:

```
/// This struct contains the account `env::signer_account_id()`, but is only
constructible given a
/// set of participants that include the signer, thus acting as a typesystem-based
enforcement
/// mechanism (albeit a best-effort one) for authenticating the signer.
#[near(serializers=[borsh, json])]
#[derive(Debug, Clone, PartialEq, Eq, PartialOrd, Ord, Hash)]
pub struct AuthenticatedAccountId(AccountId);
impl AuthenticatedAccountId {
    pub fn get(&self) -> &AccountId {
        &self.0
    }
    pub fn new(participants: &Participants) -> Result<Self, Error> {
        let signer = env::signer_account_id();
        if participants
            .participants()
            .iter()
            .any(|(a_id, _, _)| *a_id == signer)
        {
```

```
            Ok(AuthenticatedAccountId(signer))
        } else {
            Err(InvalidState::NotParticipant.into())
        }
    }
}
```

Figure 4.1: Account ID object initialization (mpc/crates/contract/src/primitives/key_state.rs#167–189)

Five public vote methods rely on these unsafe-by-default primitives without an upstream `voter_or_panic()` or `assert_caller_is_attested_participant_and_protocol_active()` guard, so each of them can have a vote cast through the participant's identity by any contract the participant signs a transaction for:

- `vote_new_parameters`
- process_new_parameters_proposal
- `vote_foreign_chain_policy`
- `vote_add_domains`
- `vote_cancel_resharing`
- `vote_cancel_keygen`

The `vote_new_parameters` endpoint is by far the most powerful, allowing someone who can trick a quorum of the old set to entirely choose the new set: either by installing new malicious parties, or by simply shrinking the committee to only participants they have influence over.

It appears that these access controls were implemented piecemeal. PR #1316 added validation for some important scenarios but did not add the check to these methods, and PR #1961 added `vote_foreign_chain_policy` without the check.

Exploit Scenario

Bob runs a node for the NEAR foreign transaction service, but he has weak security practices. The current threshold is at least `MIN_THRESHOLD_ABSOLUTE+1`, and Alice controls `t-1` committee members. She tricks Bob into calling her malicious contract in a transaction, which then triggers `vote_new_parameters` on a proposal to reduce the committee to exactly Alice's nodes, and reduce the threshold to `t-1`.

Recommendations

Short term, add the checks to each entrypoint.

Long term, consider adding this check to core functions, or using “permission objects” to ensure that the API cannot perform certain actions without checking permissions. For example, if you have a private newtype `PredecessorCheck` whose only constructor calls `assert_caller_is_attested_participant_and_protocol_active`, and an internal helper function such as `AuthenticatedAccountId::new()` that requires `&PredecessorCheck`, implementing them without sufficient permissions will result in a type error.

5. Node provider ordering depends only on request ID and local configuration quirks

Severity: **Medium**	Difficulty: **Low**
Type: Configuration	Finding ID: TOB-NEARFTV-5
Target: `crates/node/src/providers/verify_foreign_tx/sign.rs`	

Description

In order to spread RPC requests among different provider URLs, the specification says that the providers should be sorted according to a hash of the participant ID, request ID, and RPC URL, as shown below in figure 5.1.

````
## Deterministic Provider Selection

Each node selects a provider using a deterministic hash of the policy identity
(provider RPC URL):

```
hash = sha256(participant_id || request_id || provider_rpc_url)
```

Providers are sorted by this hash to build a deterministic ordering:

* **Primary provider** = first in the ordering.

This ensures different nodes query different providers for the same request while
preserving determinism.
````

Figure 5.1: The specification for deterministic provider selection (`mpc/docs/foreign-chain-transactions.md#321–333`)

However, in the code, only the request URL is used, and the ordering is based on a shuffle, which then indexes into the collection of providers depending on their order, sorted by the configuration-name of each provider.

```
/// Deterministically selects a provider index based on the request ID and the
node's
/// position within the participant set.
///
/// Uses the request ID as a seed to create a deterministic permutation of provider
indices,
/// then selects the index at position `my_participant_index % num_providers`. This
ensures
```

```
/// that RPC selection is balanced.
fn select_provider(
    num_providers: usize,
    request_id: &CryptoHash,
    my_participant_index: usize,
) -> Option<usize> {
    if num_providers == 0 {
        return None;
    }
    let mut indices: Vec<usize> = (0..num_providers).collect();
    let mut rng = rand::rngs::StdRng::from_seed(request_id.0);
    indices.shuffle(&mut rng);
    Some(indices[my_participant_index % num_providers])
}
```

Figure 5.2: Provider selection is implemented differently from the specification. (mpc/crates/node/src/providers/verify_foreign_tx/sign.rs#353–371)

```
let provider_index = select_provider(
    bitcoin_config.providers.len(),
    &request_id,
    my_participant_index,
);

let bitcoin_provider_config =
    provider_index.and_then(|i| bitcoin_config.providers.values().nth(i));
```

Figure 5.3: The provider index refers to the nth value in a `BTreeMap`, which is ordered by configuration-local labels for each provider. (mpc/crates/node/src/providers/verify_foreign_tx/sign.rs#159–166)

This means that the provider ordering is both somewhat nondeterministic and based on an identical seed for each participant. We did not confirm whether this has meaningful security implications, since the current splitting behavior appears to be primarily for load balancing, and the specification already states the risk that a "malicious or faulty provider could return incorrect data for a subset of nodes."

Exploit Scenario

Alice runs a malicious provider. When the request ID is correct, all providers go to her node and confirm an incorrect result.

Recommendations

Short term, implement the spec's participant ID selection.

Long term, ensure that all specified behavior of the system has testing that confirms that the implementation matches the specification.

6. API tokens always have a prefix in authentication headers (defaults to “Bearer”)

Severity: **Informational**	Difficulty: **Not Applicable**
Type: Data Validation	Finding ID: TOB-NEARFTV-6
Target: `crates/node-config/src/foreign_chains/auth.rs`, `crates/node/src/config.rs`	

Description

The handling for RPC authentication tokens that are transmitted in headers can only express formats that have a non-empty prefix word, such as “Bearer <key>”. The implementation rejects empty strings in the configuration file, and also replaces None with `Bearer` when using the keys, as shown in figures 6.1 and 6.2.

```
pub(crate) fn validate_auth_config(
    auth: &AuthConfig,
    rpc_url: &str,
    chain_label: &str,
    provider_name: &str,
) -> anyhow::Result<()> {
    match auth {
        AuthConfig::None => Ok(()),
        AuthConfig::Header { scheme, .. } => {
            if let Some(scheme) = scheme {
                anyhow::ensure!(
                    !scheme.trim().is_empty(),

"foreign_chains.{chain_label}.providers.{provider_name}.auth.scheme must be
non-empty if provided"
                );
            }
            Ok(())
        }
…
    }
}
```

Figure 5.1: Header auth validation forbids empty strings. (mpc/crates/node-config/src/foreign_chains/auth.rs#65–101)

```
/// Convert an [`AuthConfig`] into a [`foreign_chain_inspector::RpcAuthentication`].
///
/// This lives in mpc-node (rather than the config crate) to avoid adding
```

```rust
/// `foreign-chain-inspector` as a dependency of the lightweight config crate.
pub fn auth_config_to_rpc_auth(
    auth: AuthConfig,
    rpc_url: &mut String,
) -> anyhow::Result<foreign_chain_inspector::RpcAuthentication> {
    match auth {
        AuthConfig::None => Ok(RpcAuthentication::KeyInUrl),
        AuthConfig::Header {
            name: header_name,
            scheme,
            token,
        } => {
            let scheme = scheme.unwrap_or_else(|| "Bearer".to_string());
            let token_value = token.resolve()?;
            let header_value = HeaderValue::from_str(&format!("{scheme}
{token_value}"))?;
            Ok(RpcAuthentication::CustomHeader {
                header_name,
                header_value,
            })
        }
        AuthConfig::Path { placeholder, token } => {
            let token_value = token.resolve()?;
            *rpc_url = rpc_url.replace(&placeholder, &token_value);
            Ok(RpcAuthentication::KeyInUrl)
        }
        AuthConfig::Query { name: _, token: _ } => anyhow::bail!("this is not
supported yet"),
    }
}
```

Figure 5.2: A None scheme becomes "Bearer <key>" (mpc/crates/node/src/config.rs#444–474)

Recommendations

Short term, add a case for no-prefix tokens.

Long term, ensure that all reasonable token formats intended for the system's use are enumerated and tested.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

From September 9 to September 10, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the NEAR One team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

NEAR One has resolved all six issues in this report. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	Duplicate requests overwrite pending yield promises	Medium	Resolved
2	MpcContract::respond_verify_foreign_tx does not check that payload_hash matches the response	High	Resolved
3	verify_foreign_transaction does not charge properly for storage	Low	Resolved
4	Several voting methods accept cross-contract calls under the participant's signer identity	High	Resolved
5	Node provider ordering depends only on request ID and local configuration quirks	Medium	Resolved
6	API tokens always have a prefix in authentication headers (defaults to "Bearer")	Informational	Resolved

Detailed Fix Review Results

TOB-NEARFTV-1: Duplicate requests overwrite pending yield promises
Resolved in commit `3f2d18f`. Pending requests now append each yield index to a bounded queue instead of replacing the prior value for an identical request key, and a response drains and resumes every queued yield. Timeout cleanup removes one oldest queue entry at a time, so one caller's timeout does not orphan its siblings. Tests cover identical requests from different callers and confirm that both receive the same response.

TOB-NEARFTV-2: MpcContract::respond_verify_foreign_tx does not check that payload_hash matches the response
Resolved in commit `c2e8ed5`. Requests may now carry an optional expected payload hash, which the SDK derives from the expected extracted values, nodes check before signing, and the contract compares with the response hash. We note that the field defaults to `None` for compatibility, and both node and contract checks are conditional on it being present. Legacy callers that omit the field therefore retain the original unbound behavior.

TOB-NEARFTV-3: verify_foreign_transaction does not charge properly for storage
Resolved in commit `8b20426`. Every foreign-chain request now stores its extractors in an `UpperBoundedVec` capped at 32 entries, which bounds overall request size. The `verify_foreign_transaction` function also requires a one-yoctoNEAR deposit to mitigate storage exhaustion attacks.

TOB-NEARFTV-4: Several voting methods accept cross-contract calls under the participant's signer identity
Resolved in commits `d779621`, `f83dd6d`, and `7555fb7`. The `vote_foreign_chain_policy` entrypoint was removed, and its replacement `vote_update_foreign_chain_providers` checks the predecessor through `voter_or_panic`.

Direct assertions (through `assert_caller_is_signer`) also guard the `vote_new_parameters` `vote_add_domains`, `vote_cancel_resharing`, and `vote_cancel_keygen` handlers. The `process_new_parameters_proposal` function is internal and is reachable only through guarded `vote_new_parameters`. The lower-level authenticated-ID constructors still only check signer identity. However, every affected public call path now rejects forwarded cross-contract calls.

TOB-NEARFTV-5: Node provider ordering depends only on request ID and local configuration quirks
Resolved in commit `ef92892`. Each request now queries all configured providers concurrently and rejects conflicting results. Tests confirm that mismatched results are rejected correctly.

TOB-NEARFTV-6: API tokens always have a prefix in authentication headers (defaults to "Bearer")
Resolved in commit `e0bf42a`. Header authentication now only prepends the scheme if it is present, defaulting to the raw token. Two unit tests ensure that both cases are handled correctly.

C. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	**Description**
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to NEAR One under the terms of the project statement of work and has been made public at NEAR One's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the Trail of Bits Publications page. Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.